

```
import numpy as np

PI = np.pi

def int_to_bits_lsb(value: int, length: int) -> np.ndarray:

    if value < 0:
        raise ValueError("value >= 0 must be")
    if value >= (1 << length):
        raise ValueError("value size not combinian into import length")

    return np.array([(value >> i) & 1 for i in range(length)],
                    dtype=np.uint8)

class MSK_mapper:

    def __init__(self, bit_rate, signal_energy=1, h=0.5):

        self._h          = h
        self._bit_rate    = bit_rate
        self._T_symb      = 1/self._bit_rate
        self._E           = signal_energy

    def _get_theta_n(self, bit_sequence, n):

        if (n >= np.size(bit_sequence)): raise ValueError("n >= size of bit sequence")

        if n == 0: return 0

        theta = 0

        for i in range(0, n):
            theta += (2*bit_sequence[i] - 1)

        theta_n = PI*self._h*theta

        return theta_n

    def get_average_energy(self):

        return self._E

    def get_baseband_signal(self, bit_sequence):

        def baseband_signal_t(t):
            n = int(t // self._T_symb)
            theta_n = self._get_theta_n(bit_sequence, n)

            phi_t = theta_n + (PI*self._h*(2*bit_sequence[n] - 1)) * (t - n*self._T_symb)/self._T_symb

            return np.sqrt(2*self._E/self._T_symb)*np.exp(1j*phi_t)

        def baseband_signal(t_arr):

            if np.isscalar(t_arr):
                return baseband_signal_t(t_arr)

            result = np.zeros_like(t_arr, dtype=complex)

            for i in range(0, np.size(t_arr)):
                result[i] = baseband_signal_t(t_arr[i])

            return result

        return baseband_signal

    def get_modulate_sequence_and_time_and_phase(self, t_start, t_end, numb_of_points, bit_sequence):

        if (t_end - t_start) <= 0: raise ValueError("t_end <= t_start")
        if ((t_end - t_start) // self._T_symb + 1) > np.size(bit_sequence): raise ValueError("PËnsufficient bits to form the baseband signal")
        if numb_of_points <= 0: raise ValueError("numb_points must be > 0")

        t = np.linspace(t_start, t_end, numb_of_points)
        result = np.zeros(numb_of_points, dtype=complex)
        phases = np.zeros(numb_of_points, dtype=complex)

        theta_n = 0
        cur_n = 0

        for i in range(0, numb_of_points):
            n = int((t[i] - t[0]) // self._T_symb)

            if cur_n != n:
                theta_n += (2*bit_sequence[n - 1] - 1)*PI*self._h
                cur_n = n

            phi_t = theta_n + (PI*self._h*(2*(bit_sequence[n]) - 1)) * (t[i] - n*self._T_symb)/self._T_symb

            phases[i] = phi_t
            result[i] = np.sqrt(2*self._E/self._T_symb)*np.exp(1j*phi_t)

        return result, t, phases
```